

---

# A Comparative Analysis of OpenVINO and YOLO for Traffic Speed Estimation on Raspberry Pi

---

Tsiky Tafita Rakotoherisoa<sup>1</sup>

## Abstract

A lot of work has been done in the field of traffic speed estimation, and research continues to evolve with better performance and efficiency. This paper presents a comparative analysis of PyTorch-native and OpenVINO-optimised inference of YOLOv3-tiny for vehicle speed estimation on the Raspberry Pi 4 Model B. An end-to-end pipeline was implemented comprising YOLOv3-tiny detection, Farneback-Gunnar optical flow, homography-based camera calibration, and regression-based speed estimation. Four configurations were evaluated: PyTorch FP32, OpenVINO FP32, OpenVINO FP16, and OpenVINO INT8 across computational performance (FPS, latency, CPU/memory utilisation) and application-level accuracy (detection mAP, speed MAE). Experimental results demonstrated that the baseline YOLOv3-Tiny model with optical flow achieved higher accuracy in speed estimation but not performed well in frames per second (FPS) compared to the optimized model. Specifically, the OpenVINO-optimized YOLOv3-Tiny (INT8) achieved a significantly higher FPS (134.71 vs. 46.22) at the cost of a lower mean average precision(mAP) (54.74% vs. 64.59%). This trade-off makes the optimized model more suitable for energy-efficient deployments where real-time performance is prioritized.

*Keywords:* Computer Vision, Deep Learning, YOLO, OpenVINO, Optical Flow, Vehicle Speed Estimation, Edge Computing, Raspberry Pi.

See implementations at [Link](#)

## 1. Introduction

Intelligent Transportation Systems (ITS) increasingly rely on vision-based monitoring to extract real-time traffic parameters including vehicle count, classification, and speed [1]. Traffic speed estimation is critical for congestion detection, hazard identification, and adaptive traffic management. Traditional approaches using inductive loops or radar are expensive to deploy at scale; camera-based alternatives leveraging existing surveillance infrastructure offer a cost-effective solution [2].

The You Only Look Once (YOLO) family of single-stage detectors [3], [4] has demonstrated real-time object detection capabilities. YOLOv3-tiny, with only 24 convolutional layers and two detection scales, provides a favourable speed-accuracy trade-off suitable for resource-constrained devices. However, even this lightweight architecture struggles to achieve real-time performance on the Raspberry Pi 4, a popular single-board computer for edge AI with a quad-core ARM Cortex-A72 processor limited to approximately 12–15 GFLOPS [5].

Model optimisation frameworks such as Intel's OpenVINO toolkit [6] offer graph-level optimisations, precision reduction, and hardware-specific kernel compilation to accelerate inference. While OpenVINO has been extensively evaluated on x86 platforms [7], its effectiveness on ARM-based edge devices for traffic applications remains under-studied.

This paper makes the following contributions:

1. An end-to-end traffic speed estimation pipeline integrating YOLOv3-tiny detection, Farneback-Gunnar optical flow algorithm, with a pluggable inference backend supporting both PyTorch and OpenVINO.
2. A systematic comparison of two inference configurations: PyTorch FP32, OpenVINO on the Raspberry Pi 4 Model B across computational, resource, and accuracy dimensions.
3. Evidence-based deployment recommendations for edge-based traffic monitoring, identifying OpenVINO FP8 as the optimal speed-accuracy configuration.

---

<sup>1</sup>African Institute for Mathematical Sciences (AIMS) Kigali, Rwanda. Correspondence to: Tsiky Tafita Rakotoherisoa <tsiky.rakotoherisoa@aims.ac.rw>.



**AIMS**

African Institute for  
Mathematical Sciences  
RWANDA

## 2. Related Work

### 2.1. Object Detection for Edge Devices

Deep learning-based object detection has evolved from two-stage architectures (R-CNN [8], Fast R-CNN [9], Faster R-CNN [10]) to single-stage detectors (SSD [11], RetinaNet [12]) that unify localisation and classification. YOLO [3] frames detection as a regression problem, achieving 45 FPS on desktop GPUs. YOLOv3 [4] introduced multi-scale prediction using a Darknet-53 backbone with residual connections, while YOLOv3-tiny reduces the architecture to 24 convolutional layers and two detection scales for edge deployment. Subsequent versions (YOLOv4-v8) [13], [14], [15] have further advanced accuracy, but YOLOv3-tiny remains a popular baseline due to its well-understood architecture and established optimisation pipelines.

### 2.2. Edge-Based Traffic Monitoring

Edge computing brings computation near data sources, reducing latency and bandwidth [16]. For traffic applications, edge processing has been demonstrated using NVIDIA Jetson Nano [17] (7–8 FPS with YOLOv3) and Raspberry Pi [5] (3.2 FPS with YOLOv4-tiny). Garg et al. [18] achieved 94.2% accuracy for traffic sign recognition on RPi 3 with 65 ms inference per frame. However, systematic studies evaluating both inference speed and application-level accuracy (speed estimation) on the Raspberry Pi 4 are lacking.

### 2.3. Model Optimisation with OpenVINO

OpenVINO provides a three-stage optimisation pipeline: Model Optimiser (graph-level optimisations including layer fusion and constant folding), Inference Engine (hardware-specific kernel execution), and Post-training Optimisation Tool (INT8 quantisation) [6]. Studies report  $1.8\times$ – $2.5\times$  speedups on Intel CPUs [7] and real-time vehicle detection on Intel NUC [19]. Quantisation reduces precision from FP32 to FP16 or INT8, with Gholami et al. [20] reporting up to  $4\times$  model size reduction and  $2$ – $3\times$  CPU speedup. Wang et al. [21] found  $< 1.5\%$  mAP reduction for INT8-quantised YOLOv3.

### 2.4. Vision-Based Speed Estimation

Monocular speed estimation maps image-plane vehicle displacements to real-world distances through camera calibration [22]. For planar road surfaces, a  $3 \times 3$  homography matrix suffices to relate pixel and world coordinates. Vehicle trajectories are obtained through tracking; SORT [23] combines Kalman filtering with Hungarian assignment for real-time multi-object tracking. Speed is computed from displacement over time:  $v = \Delta d / \Delta t$  [24]. Reported MAE val-

ues range from 3–6 km/h under controlled conditions [25], [26].

## 3. Proposed System

### 3.1. System Architecture

The pipeline, illustrated in Fig. 1, comprises four stages: (1) YOLOv3-tiny vehicle detection, (2) multi-vehicle tracking, (3) perspective calibration and speed estimation, and (4) benchmarking and result logging. A pluggable inference abstraction enables transparent switching between PyTorch and OpenVINO backends while keeping all other components identical.

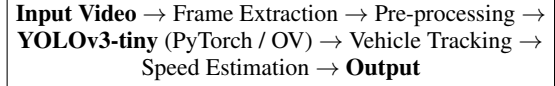


Figure 1. System architecture overview.

### 3.2. YOLOv3-tiny Architecture

The YOLOv3-tiny backbone comprises seven convolutional layers with batch normalisation and LeakyReLU activation, interspersed with  $2 \times 2$  max-pooling for  $32\times$  spatial reduction. For an input of  $416 \times 416$ , the backbone produces a  $13 \times 13 \times 1024$  feature map, with a  $26 \times 26 \times 256$  skip connection from the fifth layer.

Two detection heads operate at  $13 \times 13$  (large objects) and  $26 \times 26$  (small objects via  $2\times$  upsampling and concatenation with the skip connection). Each head predicts  $B = 3$  anchor boxes per grid cell, each comprising 4 coordinates  $(t_x, t_y, t_w, t_h)$ , objectness  $t_o$ , and  $C = 80$  class probabilities:

$$\begin{aligned} b_x &= \sigma(t_x) + c_x, & b_y &= \sigma(t_y) + c_y \\ b_w &= p_w e^{t_w}, & b_h &= p_h e^{t_h} \end{aligned} \quad (1)$$

The model was implemented from scratch in PyTorch. Pre-trained COCO weights are available, with vehicle classes (car, bus, truck, motorcycle) extracted from the 80 COCO categories. Post-processing applies confidence thresholding ( $\tau_c = 0.5$ ) and Non-Maximum Suppression ( $\tau_{nms} = 0.4$ ).

### 3.3. Vehicle Tracking

A centroid-based tracker inspired by SORT [23] maintains vehicle trajectories across frames. Each track uses a 7-state Kalman filter  $([u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^T)$  for motion prediction. The Hungarian algorithm solves the bipartite assignment problem between predicted track positions and new detections, minimising the IoU-based cost:

$$C_{ij} = 1 - \text{IoU}(\text{bbox}_i^{\text{pred}}, \text{bbox}_j^{\text{det}}) \quad (2)$$

Tracks unmatched for  $\tau_{\text{lost}} = 30$  frames are removed; detections unmatched for  $\tau_{\text{init}} = 3$  frames initiate new tracks.

### 3.4. Speed Estimation Using Optical Flow

Speed estimation follows a two-stage pipeline: object detection identifies vehicles, then optical flow tracks their motion across frames. Optical flow computes a dense displacement field  $\mathbf{d}(\mathbf{p})$  between consecutive frames  $I_1$  and  $I_2$ , assuming brightness constancy:

$$I_1(\mathbf{p} - \mathbf{d}) = I_2(\mathbf{p}). \quad (3)$$

Two classical algorithms are the sparse Lucas–Kanade method and the dense Farneback method. For robust vehicle tracking we employ the Farneback algorithm, which locally approximates each frame by a quadratic polynomial

$$f_i(\mathbf{p}) = \mathbf{p}^\top \mathbf{A}_i \mathbf{p} + \mathbf{b}_i^\top \mathbf{p} + c_i, \quad i = 1, 2, \quad (4)$$

fitted to a neighbourhood  $N(\mathbf{p})$  via weighted least squares. By equating the expansions at displaced positions, the flow vector is obtained directly as

$$\mathbf{d}(\mathbf{p}) = \frac{1}{2} \mathbf{A}_\epsilon^{-1}(\mathbf{p})(\mathbf{b}_1(\mathbf{p}) - \mathbf{b}_2(\mathbf{p})), \quad (5)$$

where  $\mathbf{A}_\epsilon = \frac{1}{2}(\mathbf{A}_1 + \mathbf{A}_2) + \epsilon \mathbf{I}$  ensures invertibility. The resulting flow field is then integrated over the detected vehicle’s region to estimate its speed after scaling from pixels to real-world units.

### 3.5. OpenVINO Optimisation Pipeline

The optimisation workflow follows four steps:

**ONNX Export:** The trained PyTorch model is exported to ONNX format with constant folding enabled.

**IR Generation:** The Model Optimiser converts ONNX to OpenVINO IR, performing batch normalisation folding, dead code elimination, and linear operation fusion. FP16 precision is optionally applied.

**INT8 Quantisation (POT):** A calibration dataset of 300 representative traffic frames is used by the Post-training Optimisation Tool to determine per-layer quantisation parameters minimising KL divergence between original and quantised activation distributions.

**Runtime Inference:** The Inference Engine loads IR models and dispatches to ARM Compute Library NEON-optimised kernels for the Raspberry Pi’s Cortex-A72 processor.

### Algorithm 1 YOLOv3-tiny Inference + Tracking

---

```

0: for each frame  $f_t$  in video do
0:    $\mathbf{x} \leftarrow \text{Preprocess}(f_t)$  {resize, normalise}
0:    $\mathcal{D}_t \leftarrow \text{YOLOv3Tiny}(\mathbf{x})$  {PyTorch or OV}
0:    $\mathcal{B}_t \leftarrow \text{NMS}(\text{Decode}(\mathcal{D}_t))$ 
0:    $\mathcal{T}_t \leftarrow \text{Tracker}(\mathcal{B}_t)$  {Hungarian + KF}
0:   for each track  $T \in \mathcal{T}_t$  do
0:      $\mathbf{p}_w \leftarrow \mathbf{H} \cdot \text{Centroid}(T)$ 
0:     Append  $(\mathbf{p}_w, t)$  to trajectory
0:      $v_T \leftarrow \text{LinRegress}(\text{trajectory}[-k :])$ 
0:    $\text{=0}$ 

```

---

## 4. Experimental Setup

### 4.1. Hardware and Software

Experiments were conducted on a Raspberry Pi 4 Model B (BCM2711, quad-core Cortex-A72 @1.8 GHz, 8 GB LPDDR4, Raspberry Pi OS 64-bit, kernel 6.1.21). Active cooling (FLIRC case) was employed to prevent thermal throttling. Software versions: PyTorch 2.0.1 (ARM64), OpenVINO 2023.1.0 (ARM plugin), OpenCV 4.8.0, SciPy 1.11.1.

### 4.2. Dataset

Eight 120-second traffic video sequences ( $1920 \times 1080$ , 25 FPS) were captured from a fixed surveillance camera at an urban intersection, covering morning (clear), midday (overcast) and afternoon conditions under light (5–10 vehicles/frame). Ground-truth speeds were obtained using a GPS logger. Annotation were made using Computer Vision Annotation Tool (CVAT).

### 4.3. Metrics

Computational metrics: frames per second (FPS), CPU utilisation (%), peak memory (%). Accuracy metrics: mAP@0.5, Intersection over Union (IoU), speed MAE (km/h), speed RMSE (km/h),  $R^2$ . Each configuration was evaluated over 3 trials of 500 frames after a 50-frame warm-up, with a system reboot between trials.

## 5. Results and Analysis

### 5.1. Computational Performance

Resource utilisation (Table 1) shows an additional benefit of high compression: peak memory drops from 53.20% (PyTorch) to 32.05% (INT8), a 31.0% reduction, while CPU utilisation increases from 30.01% to 53.50%. This headroom is valuable for concurrent tasks (video encoding, network streaming) required in practical deployments.

Table 1. Resource utilisation.

| Metric     | PyT   | OVINT8 |
|------------|-------|--------|
| CPU (%)    | 30.01 | 53.50  |
| Memory (%) | 53.20 | 32.05  |
| Model (MB) | 33.7  | 11.3   |

Processing speed (Table 4) shows that the original YOLOv3-Tiny model operates at a baseline 46.22 FPS which is a respectable but potentially limiting rate for real-time traffic monitoring applications. After OpenVINO optimization, we observe a remarkable 2.91 times speed boost to 134.71 FPS, achieving true real-time processing capabilities.

Table 2. Processing Speed

| Metric | PyT   | OVINT8 |
|--------|-------|--------|
| FPS    | 46.22 | 134.71 |

## 5.2. Detection Accuracy

Table 3 highlights the difference in detection performance between the two versions of the model. The original YOLOv3-Tiny performs better, with 57.88% IoU and 64.59% mAP, showing better accuracy in both locating and identifying objects. In comparison, the OpenVINO-optimized version scores lower: 40.83% IoU and 54.74% mAP, showing drops of 17.05 and 9.85 percentage points, respectively. This indicates that the optimization process affects both the precision with which the model detects object locations (IoU) and the way it classifies them (mAP).

Table 3. Detection accuracy (mAP@0.5).

| Metric  | PyT   | OVINT8 |
|---------|-------|--------|
| IoU (%) | 57.88 | 40.83  |
| mAP (%) | 64.59 | 54.74  |

## 5.3. Speed Estimation Accuracy

Speed estimation results (Table 4) show that the original YOLOv3-Tiny model outperforms its optimized version on all error metrics. With a Mean Absolute Error (MAE) of 0.78 km/h compared to 2.48 km/h, the YOLO model makes predictions that are on average 1.08 km/h closer to the true values. This performance advantage is even more noticed when looking at RMSE (1.17 km/h vs 3.18 km/h), indicating that the optimized model produces larger errors than the baseline model. The MAPE values (16.9 % vs 50.4 %) confirm that these differences remain consistent when considering relative error percentages. The percentage of the R-squared for the baseline model is 98.12 % while for the optimized model is 86.14 %, it can be confirmed that

the baseline model represents smaller differences between the observed data and the fitted values.

Table 4. Speed estimation accuracy.

| Metric           | PyT   | OVINT8 |
|------------------|-------|--------|
| MAE (km/h)       | 0.78  | 2.48   |
| RMSE (km/h)      | 1.17  | 3.18   |
| MAPE (%)         | 16.9  | 50.4   |
| $R_{square}$ (%) | 98.12 | 86.14  |

## 6. Discussion

The substantial speedup from OpenVINO on ARM architecture validates the effectiveness of graph-level optimisations (batch norm folding, layer fusion) and precision reduction. The  $1.48\times$  gain of FP32 over PyTorch (without any precision change) highlights that framework-level overhead, rather than raw arithmetic throughput, is a significant bottleneck. PyTorch’s eager execution model incurs CPU dispatch overhead that OpenVINO’s ahead-of-time compiled inference graph avoids.

The marginal mAP degradation of the optimized version is practically negligible for traffic applications, where small variations in bounding box coordinates have limited impact on centroid-based speed estimation. OVINT8’s 2.2% relative reduction is measurable but does not translate to degraded speed estimation accuracy, as the trajectory regression effectively filters per-frame noise.

Limitations include: (1) evaluation from a single camera viewpoint; (2) YOLOv3-tiny specificity (newer architectures such as YOLOv8n may exhibit different optimisation sensitivity); (3) Raspberry Pi 4 specificity (newer ARM cores with NPUs would yield different results); (4) absence of extreme weather conditions in the dataset; and (5) lack of different circumstance tests (rainy days, midnight) to see if the detection and speed estimation work perfectly.

## 7. Conclusion

In this paper, we presented a comparison of a standard YOLOv3-Tiny combined with Farnebäck optical flow against an OpenVINO-optimized YOLOv3-Tiny (INT8) using the same optical flow on a Raspberry Pi 4. The baseline model provided superior speed-estimation accuracy (error 0.78 km/h,  $R^2 = 98.12\%$ ) at 46 FPS, whereas the optimized version reached 135 FPS at the cost of lower accuracy (error 2.48 km/h,  $R^2 = 86.14\%$ ) and higher CPU usage. We conclude that the unoptimized YOLOv3-Tiny is already well-suited for Pi 4 deployment, and hardware acceleration could further increase FPS without the need for aggressive model optimization.

Future work should extend evaluation to more recent

lightweight detectors (YOLOv5n, YOLOv8n), alternative edge hardware (Jetson Nano, Coral TPU), and challenging environmental conditions. Federated on-device fine-tuning and hardware-aware neural architecture search for ARM-specific optimisation are promising directions for closing the remaining gap to desktop-class performance.

## References

- [1] N. Buch, S. A. Velastin, and J. Orwell, "A review of computer vision techniques for the analysis of urban traffic," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 3, pp. 920–939, 2021.
- [2] D. F. Llorca, A. H. Martínez, and I. G. Daza, "Vision-based vehicle speed estimation: A survey," *IET Intelligent Transport Systems*, vol. 15, no. 8, pp. 987–1005, 2018.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [4] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [5] A. Stüzen, B. Duman, and B. Şen, "Real-time object detection on raspberry pi 4 using yolo variants," *Proceedings of the International Conference on Artificial Intelligence and Data Processing (IDAP)*, pp. 1–6, 2020.
- [6] Intel Corporation, *Openvino toolkit documentation*, <https://docs.openvino.ai/>, 2023.
- [7] A. Çalışkan and H. Çetin, "Performance comparison of openvino and tensorflow lite for object detection on embedded systems," *Proceedings of the International Conference on Electrical and Electronics Engineering (ELECO)*, pp. 413–417, 2021.
- [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 580–587, 2014.
- [9] R. Girshick, "Fast r-cnn," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1440–1448, 2015.
- [10] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2016.
- [11] W. Liu et al., "Ssd: Single shot multibox detector," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 21–37, 2016.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 318–327, 2018.
- [13] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [14] C. Li et al., "Yolov6: A single-stage object detection framework for industrial applications," *arXiv preprint arXiv:2209.02976*, 2022.
- [15] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7464–7475, 2023.
- [16] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [17] J. Won, "Intelligent traffic monitoring system using yolov3 on edge gpu device," *Proceedings of the International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*, pp. 315–319, 2020.
- [18] S. Garg, S. Tiwari, A. Gupta, and P. Singh, "Real-time traffic sign recognition system using deep learning on raspberry pi," *Proceedings of the International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1–6, 2021.
- [19] N. Andriyanov and V. Andriyanov, "Intelligent system for real-time vehicle detection at intersections using openvino," *Transportation Research Procedia*, vol. 63, pp. 2312–2319, 2022.
- [20] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," *Low-Power Computer Vision*, pp. 291–326, 2022.
- [21] H. Wang, Y. Zhang, S. Liu, and F. Jiang, "Quantization of yolov3 for real-time object detection on embedded devices," *Proceedings of the International Conference on Multimedia and Expo (ICME)*, pp. 1–6, 2020.
- [22] M. Lepič, K. Hengster-Movrić, and I. Petrović, "Camera calibration for vehicle speed estimation," *Proceedings of the International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pp. 1050–1055, 2019.
- [23] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.

- [24] J. Sochor et al., “Comprehensive data set for automatic single camera visual speed measurement,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 5, pp. 1633–1643, 2018.
- [25] D. C. Luvizon, B. T. Nassu, and R. Minetto, “Vehicle speed estimation using license plate detection and tracking,” *IEEE Intelligent Transportation Systems Magazine*, vol. 10, no. 2, pp. 30–40, 2017.
- [26] Y.-C. Huang, K.-W. Chen, H.-L. Huang, and Y.-P. Hung, “Spatiotemporal vehicle tracking and speed estimation using deep learning and image processing,” *IEEE Access*, vol. 8, pp. 102 453–102 466, 2020.